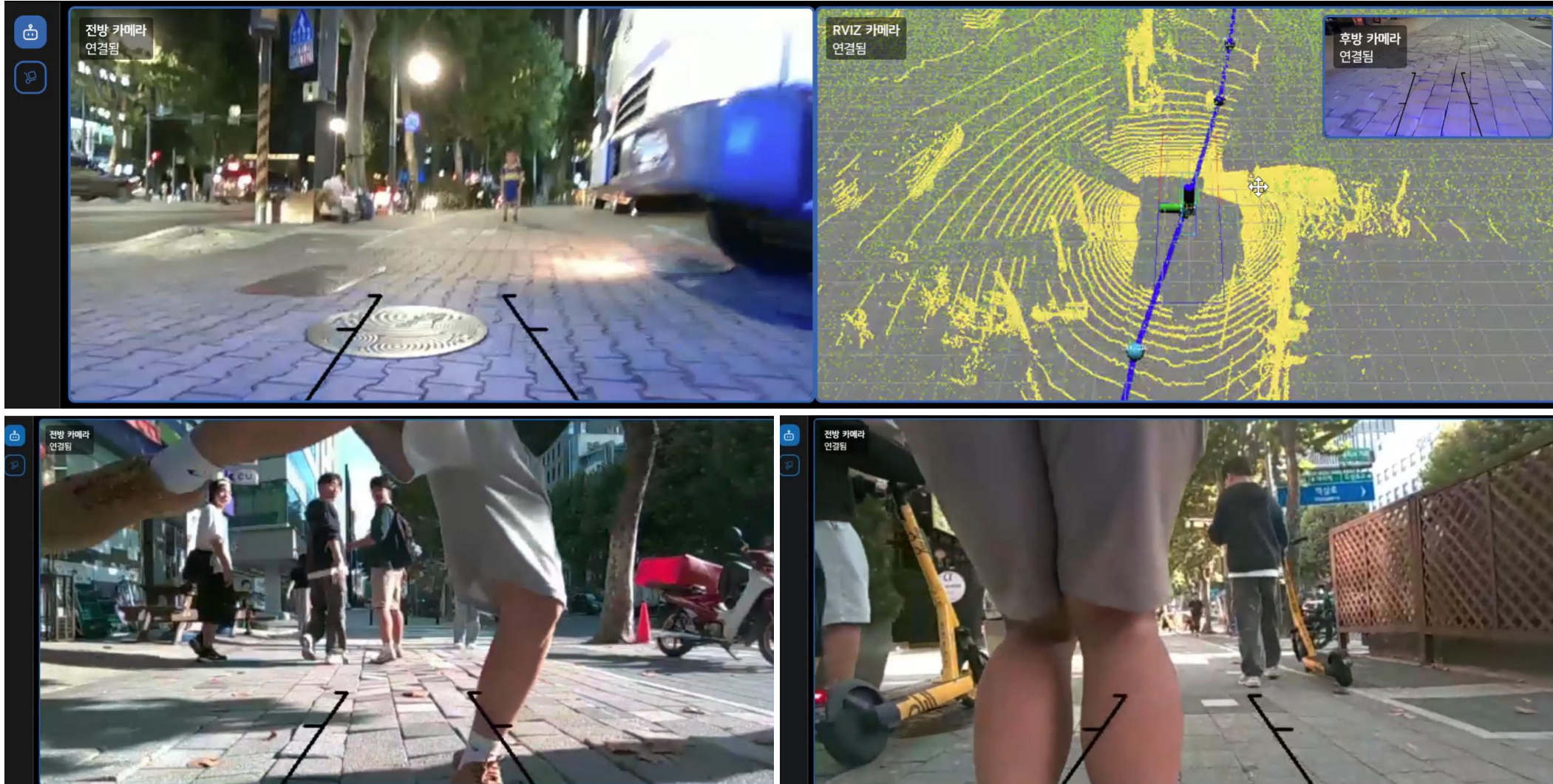


실외 자율주행 로봇 안전 주행을 위한 LiDAR Detection 연구

[개인 프로젝트]



🚨 실제 배달 중 차량·보행자·학생들로 인한 로봇 경로 방해 및 충돌 위험 빈번 발생 (대치동 지역 특성 영향)

: 현재는 사운드 알림으로 임시 대응 중이나, 근본적 해결을 위해 "LiDAR 기반 Detection 기술" 개발 필요

❖ LiDAR (Light Detection and Ranging)

- 레이저를 쏘고 반사하는 시간을 측정해 3차원 공간의 점군(point cloud)을 얻는 기술
- 이 점군(point cloud)을 활용해서 물체를 탐지(Detection), 분류(Classification), 추적(Tracking) 하는 것이 LiDAR Detection의 핵심!

▪ Object Detection

: 자동차, 보행자, 킥보드 등 객체 위치와 크기 파악 → 3D Bounding Box 표시

▪ Semantic Segmentation

: 점단위로 도로, 건물, 나무, 사람과 같은 Class 부여

▪ Instance Segmentation

: 같은 Class 내 개별 객체 분리 ex. 보행자 중 여러 명 구분

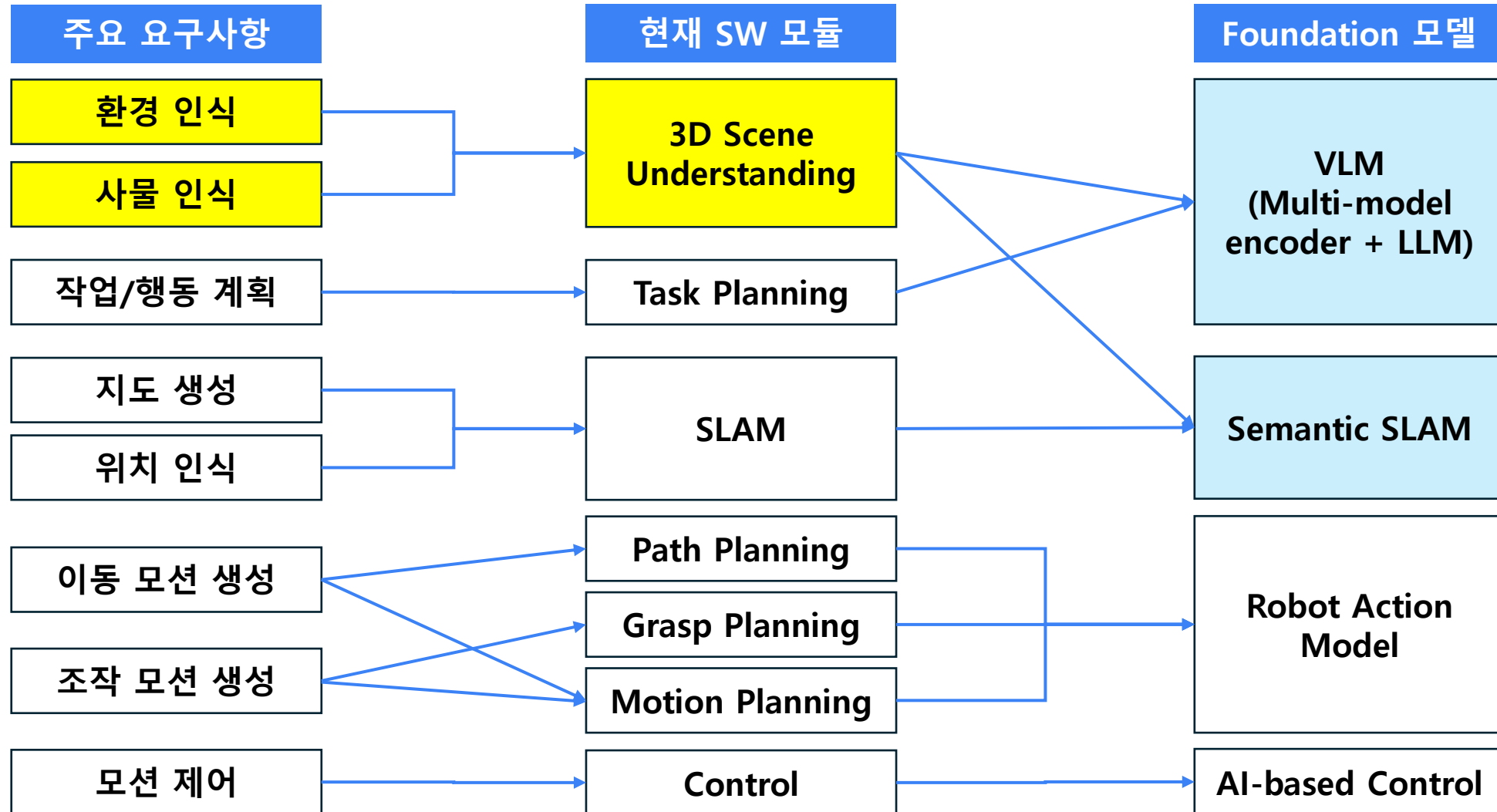
▪ Multi-sensor Fusion

: LiDAR + Camera/IMU 데이터 결합으로 성능 향상

➔ 즉, LiDAR Detection은 "안전한 자율주행" + "로봇 내비게이션을 위한 시각 인지 기술"

❖ 주요 AI 기술의 발전 방향은 유사하나, 상이한 요구사항 최적화를 위해 별도 아키텍처 채용

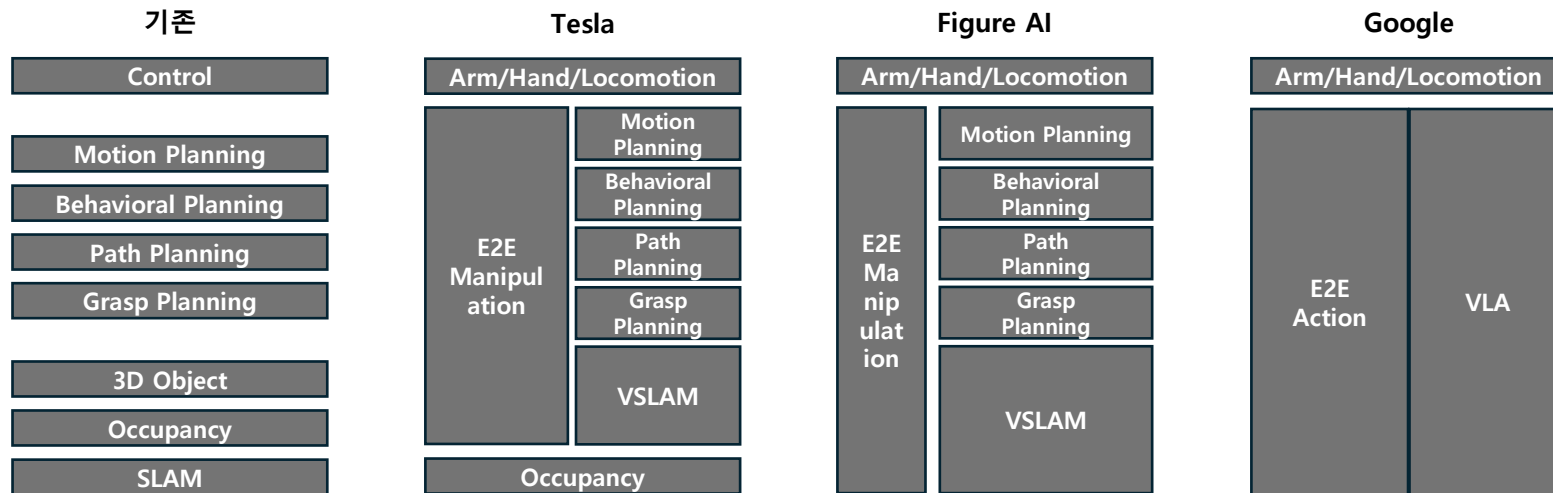
- 모듈화 구조 → E2E 시스템 → Foundation 모델기반 E2E 시스템으로 발전 중



❖ 모듈을 통합 및 최적화 하는 E2E기반 방식과 LLM/VLM 기반 일반화 연구 확산

- E2E : 내부 모듈을 신경망 기반으로 통합하여 대규모 데이터 기반의 학습으로 전체 시스템 최적화
- Multi-Task : 단일 작업 고속 실행 → 다양한 작업을 수행할 수 있는 Skill Capability 확장
- Generalization : 비정형 환경 및 배우지 않은 새로운 환경에서도 적합한 Task 수행

Reference	Methods			
	E2E	Multi-task	LLM/VLM 활용	Robot Foundation Model
Tesla Optimus	○	○	○	△
Figure AI	○	△	○	X
Google	○	○	○	○
Mobile Aloha	○	X	X	X



→ 대규모 데이터 기반 단계적 작업 성능 고도화

→ 전통 기술 기반 구축 후, E2E AI 기술 도입 중

→ 일반화 선행 연구에 집중

MOBINN은?

0단계. 기본기

-  C++ 문법 복습 (클래스, 포인터, STL, 멀티스레딩 기초)
-  Git/Github 기본 & CI/CD 개념
-  Cmake & Colcon 빌드 구조 이해

1단계. ROS2 기본기

-  ROS2 C++ 노드 작성 (rclcpp)
-  Publisher / Subscriber 패턴 실습
-  메시지 다루기
 -  sensor_msgs/PointCloud2 (LiDAR 입력)
 -  geometry_msgs/Twist (제어 명령)
 -  vision_msgs/Detection3DArray (출력 결과)
-  RViz2 시각화 (PointCloud 표시)

2단계. LiDAR & Point Cloud 처리

-  PCL (Point Cloud Library) 설치 및 실습
 -  VoxelGrid (다운샘플링)
 -  PassThrough (ROI 필터링)
 -  RANSAC (Ground 제거)
-  좌표 변환 (tf2 + Eigen)
-  전처리 파이프라인 구현 (ROS2 Node)

3단계. 딥러닝 & PointPillars 구조 이해 (Python)

- CNN, SSD 기본 구조 복습
- PointPillars 논문 정리
 - Point → Pillarization (BEV 변환)
 - Pillar FeatureNet
 - Backbone 2D CNN
 - Detection Head (cls + bbox regression)
- PyTorch 오픈소스 PointPillars 실행 (KITTI Dataset)
- Inference 결과 시각화 (Bounding Box 확인)

4단계. C++ 추론 연동

- PyTorch 학습 모델 → ONNX 변환
- C++에서 ONNX Runtime / LibTorch 로드
- ROS2 노드 통합
 - 입력: /lidar/points
 - 출력: /detected_objects
- RViz2에서 3D Bounding Box 시각화

5단계. 로봇 제어 로직 연결

- Detection 결과 파싱 (객체 클래스, 거리 계산)
- 긴급정지 로직 구현

조건: if (object == "truck" && distance < threshold) → /cmd_vel = STOP
- 테스트 (시뮬레이션 / 실제 로봇 환경)

6단계. 최적화 & 배포

- TensorRT 최적화 (Jetson 환경 대비)
- Docker 환경 세팅 (ROS2 + 딥러닝 모델 포함)
- GitHub Actions (CI/CD 자동 빌드/테스트)